

Lossless Coding of Airborne LiDAR Attributes in a Geometry-Derived Order

September 18, 2026

Abstract

Six tenths of the bits in an airborne LiDAR file belong to the attributes, not the geometry. A point cloud container decodes geometry first, so the decoder already holds every coordinate when it starts reading attributes; any order and any predictor that depends on the coordinates alone therefore costs no side information. We use that freedom to predict each attribute from its preceding nearest neighbour in space-filling curve order, with a reversible luma-chroma transform on colour. On one airborne LiDAR tile the attributes are 21% smaller than LASzip and 42% smaller than the only lossless G-PCC configuration; with the existing normalization layer the file goes from 440 MB to 295.2 MB, verified bit-exact over all points. We also derive when averaging several predictors helps, show that reordering the points is itself a coding decision worth 27% of the geometry, and quantify why the geometry side has little lossless headroom left.

1 Introduction

Point cloud compression has developed around octree occupancy coding [1, 2, 3]. G-PCC also provides the region-adaptive hierarchical transform [4] and hierarchical neighbour prediction for attributes, but the centre of gravity of the research has consistently been geometry.

Counting the bits in real data, that centre of gravity is misplaced. Normalizing one airborne LiDAR tile and then measuring the breakdown gives 41% geometry and 59% attributes. The larger half is the thinner covered.

This paper treats lossless coding of attributes. The starting point is the following observation.

A point cloud container decodes geometry first. The decoder therefore holds every coordinate at the moment it reads attributes.

It looks self-evident, but the implication is not small. Any order and any predictor defined as a function of the coordinates alone can be computed independently and identically by encoder and decoder, so neither has to be transmitted. In other words, the order in which attributes are coded may be detached from the order of the container.

1.1 Contributions

1. We state the observation above as a proposition and give an attribute coder that uses it (§3.2). Under identical conditions it is 21% smaller than LASzip and 42% smaller than the lossless G-PCC configuration.
2. We derive a condition for how many predictors should be averaged and check it against measurement (§3.3). The condition separates the attributes into three groups correctly.
3. We show that candidate selection splits into three stages of different character, two of which must be done by measurement (§3.5). Selecting by a proxy costs 2.3 bits per point on one field, and making all three stages measured recovers 3.1 bits per point in total.
4. We quantify that the arrangement of points is itself a factor in the code length (§5.6). Reordering the same point set moves the geometry code length by 27%.
5. We show from three independent angles that little lossless headroom remains on the geometry side (§6).
6. We measure the reach of the method. For terrestrial laser scanning, returning to the acquisition representation is counterproductive, and we state the condition under which it is not (§7.3).

2 Related work

The attribute coder of G-PCC [3] places points in space-filling curve order, builds a hierarchy of levels of detail, and predicts the fine levels from the coarse ones. Our method also uses a space-filling curve, but it builds no hierarchy and takes as predictor only the preceding nearest neighbour within the same level. The handling of the colour transform also differs: the colour space conversion in the reference implementation is not reversible and therefore cannot be used in lossless coding (§5.2).

LASzip [5] compresses the point records of surveying LAS column by column, with a per-type predictor and arithmetic coding; prediction is from the immediately preceding point in storage order. Our container also uses LASzip inside.

Reversible luma-chroma transforms are known from

Malvar et al. [6] as YCoCg-R and are standard in lossless still image coding. We apply this to point cloud attributes.

Ordering by space-filling curves goes back to Morton [7].

3 Formulation

3.1 Setting

We treat a point cloud as a sequence of geometry–attribute pairs

$$P = \{(\mathbf{x}_i, \mathbf{a}_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{Z}^3, \mathbf{a}_i \in \mathbb{Z}^M \quad (1)$$

The coordinates are integers at the time of storage. We write $\mathbf{f}^{(j)} = (a_1^{(j)}, \dots, a_N^{(j)})$ for the j -th attribute stacked as a column, let the reference codec be $\mathcal{C} = (\mathcal{E}, \mathcal{D})$, and write its code length as $\ell_{\mathcal{C}}$.

3.2 A coding order derived from the geometry

Let us be explicit about the order of decoding. The container decodes geometry first, so at the moment attribute decoding begins the decoder holds the whole set of coordinates $X = (\mathbf{x}_1, \dots, \mathbf{x}_N)$.

Proposition 1 (No side information) *Any order and any predictor defined as a function of X alone can be computed independently and identically by encoder and decoder. They therefore need not be transmitted.*

We use this freedom as follows. First choose a permutation determined by X alone,

$$\pi = \Pi(X), \quad \pi : \{1, \dots, N\} \rightarrow \{1, \dots, N\} \quad (2)$$

For Π we use ascending Morton code [7]. Here $\pi(t)$ is the index of the point that comes t -th in coding order, and the inverse permutation $r = \pi^{-1}$ gives the rank of a point. Next, among the points of lower rank than the current one, we take the nearest as predictors,

$$p_k(t) = \{j : r(j) < t\}, \text{ the } k \text{ nearest} \quad (3)$$

and what we send is the attribute residual

$$\mathbf{e}_t = \mathbf{a}_{\pi(t)} - \left\lfloor \frac{1}{k} \sum_{j \in p_k(t)} \mathbf{a}_j \right\rfloor \quad (4)$$

Since both Π and p_k are functions of X alone, Proposition 1 says that no side information arises.

Decoding solves (4) in the order $t = 0, 1, \dots, N - 1$. Because (3) imposes $r(j) < t$, the predictor values are already determined, so sequential decoding goes through.

The conventional scheme we compare against takes the immediately preceding point in storage order as predictor; that is the special case $\Pi = \text{id}$, $p_1(t) = \{t - 1\}$.

3.3 How many predictors

How to choose k is decided by the nature of the attribute. Suppose the attribute is a smooth field g with independent noise added,

$$a_i = g(\mathbf{x}_i) + \varepsilon_i, \quad \mathbb{E}[\varepsilon_i] = 0, \text{Var}[\varepsilon_i] = \sigma^2 \quad (5)$$

with g Lipschitz continuous with constant Λ . Using the single nearest neighbour, with distance $d_{(1)}$,

$$\mathbb{E}[e^2] \leq \Lambda^2 d_{(1)}^2 + 2\sigma^2 \quad (6)$$

and using the mean of k neighbours, with mean distance $\bar{d}_{(k)}$,

$$\mathbb{E}[e^2] \leq \Lambda^2 \bar{d}_{(k)}^2 + \sigma^2 \left(1 + \frac{1}{k}\right) \quad (7)$$

Averaging is therefore advantageous when

$$\sigma^2 \left(1 - \frac{1}{k}\right) > \Lambda^2 (\bar{d}_{(k)}^2 - d_{(1)}^2) \quad (8)$$

that is, when the noise dominates the spatial spread. Attributes with small σ should select $k = 1$ and those with large σ should select $k > 1$. We check this against measurement in §5.3.

Note that for an attribute with no spatial structure at all (Λ effectively infinite) no prediction helps, and coding it directly without differencing is best. We add that candidate to the family too.

3.4 A reversible colour transform

The three colour components correlate strongly, so we would like to decorrelate them before taking residuals, and the transform must be reversible. We use the integer lifting transform

$$\begin{aligned} C_o &= R - B, & t &= B + \lfloor C_o/2 \rfloor, \\ C_g &= G - t, & Y &= t + \lfloor C_g/2 \rfloor \end{aligned} \quad (9)$$

from [6]. Each stage has the form $u \leftarrow u \pm \lfloor v/2 \rfloor$ with v unchanged, so it is a bijection on $\mathbb{Z}^2$ regardless of the rounding. The composition is therefore a bijection as well, and the inverse returns exactly to the original in integer arithmetic.

Equation (9) and §3.2 act independently: the colour transform removes correlation between components, the coding order removes spatial correlation.

3.5 The candidate family and its selection

For each attribute we consider the following candidates.

- Keep it in the container (the inner codec holds the differencing)
- Send it externally as is (no differencing)
- Send it externally through (4) with $k \in \{1, 3, 5\}$

Selection is by code length, but the comparisons split into three kinds.

Table 1: Attributes only. Airborne LiDAR, 2M points, three colour components plus near infrared, four components in total. Input is left in acquisition order. All rows lossless

Method	Code length
LASzip	14.537
Ours, predictor from storage order	14.447
Ours, geometry-derived order and reversible colour transform	11.584

(i) **Between external candidates.** Sending as is and varying k all pass through the same coder. Code lengths can be compared directly and no estimation is needed.

(ii) **Between the container and the outside.** The amount by which the inner codec shrinks when an attribute is removed from the container does not equal the length of coding that attribute alone, because the codec uses context across columns. The standalone code length must therefore not be used as a proxy. We write the container without the attribute, measure the decrease, and take that as the cost. This adds one write per attribute, but using a proxy leads to wrong decisions (§5.4).

(iii) **After removing several.** Marginal costs are not additive: once one attribute has been removed, the marginal cost of another changes. We therefore finish by writing the configuration in which each externalized attribute is individually put back, comparing, and returning it to the container if that is cheaper. Even with (ii) measured, this stage still finds errors (§5.4).

Colour can take negative values and cannot be placed in the container, so it is outside the scope of (ii) and (iii).

Finally we write both the resulting configuration and the configuration in which no attribute leaves the container, and take the shorter. Because the identity configuration is among the candidates, the result never exceeds the original code length.

4 Experiments

The evaluation uses one tile of airborne LiDAR [8] (49.29M points, 440 MB, 18 fields). The inner codec $\mathcal{C}$ is LASzip [5]. For G-PCC we used reference implementation v23.0.

Every measurement is end-to-end coding, and on every run we confirm that all restored fields match the original exactly. No verification is done by sampling. The measurement machine has 16 CPU cores; no GPU and no learned model are used.

5 Results

5.1 Attribute code length

Table 1 compares against LASzip. As long as the predictor comes from storage order we are on par with

Table 2: Search for a G-PCC attribute configuration that is lossless. Airborne LiDAR, 2M points, no colour space conversion. Only one configuration was lossless

Method	QP	Code length	Max error	Lossless
RAHT	0, 1	encoding failed		
RAHT	4	38.104	1	no
RAHT	8	35.200	3	no
Predicting	0, 1	encoding failed		
Predicting	4	38.143	0	yes
Predicting	8	35.508	1	no
Lifting	0, 1	encoding failed		
Lifting	4	37.508	3	no
Lifting	8	35.348	3	no

Table 3: Code length in bits per point as a function of the number of predictors k . Airborne LiDAR, 2M points. Bold is the minimum of each row

Attribute	$k=1$	$k=2$	$k=3$	$k=5$	$k=8$
Intensity	8.870	8.709	8.670	8.663	8.678
Reflectance	9.031	8.855	8.801	8.776	8.785
Red	4.279	4.544	4.617	4.714	4.798
Near infrared	4.154	4.396	4.466	4.559	4.640
Deviation	4.650	5.002	5.538	6.133	6.683

LASzip (a difference of 0.6%); deriving the coding order from the geometry lowers it by 20.3%. The gain comes from the choice of order, not from a difference in coder performance.

5.2 Comparison with G-PCC

We searched for a lossless G-PCC attribute configuration by sweeping the quantization parameter and reconciling over all points (Table 2). Only predicting transform with $QP = 4$ was lossless; RAHT and lifting produce errors of 1 and 3 at the same QP. We confirmed the same conclusion at 8M points.

Comparing the three colour components alone, that configuration gives 11.936 and ours gives 6.430 bits per point (at 0.6M points). Among lossless methods ours is 46% smaller. Enabling the colour space conversion of the reference implementation brings it down to 6.588, but 0.593M points are displaced by ± 1 , so it cannot be used for lossless coding.

5.3 How many predictors

Table 3 is the check on condition (8). Intensity and reflectance are laser-derived, have large σ , decrease under averaging and bottom out at $k = 5$. Red and near infrared derive from orthoimagery, are smooth, have small σ , are best at $k = 1$ and degrade monotonically. The waveform deviation has no spatial structure at all, so averaging only makes it worse; in fact coding it directly without differencing is best, at 3.909 against 4.817 for differencing. The measurements divide into exactly the three groups the condition predicts.

Table 4: Selection method and code length. A whole airborne LiDAR tile (49.29M points). No new coding technique was added

Selection method	bits/pt	vs. orig.
Two candidates only	51.049	-28.5%
Extended candidates, proxy selection	51.085	-28.4%
+ greedy verification (iii)	48.776	-31.6%
+ measured marginal cost (ii)	47.908	-32.9%

Table 5: A whole airborne LiDAR tile (49.29M points, 18 fields). All rows restored with geometry bit-exact and attributes matching exactly

Configuration	Size	bits/pt	Red.
Original LAZ	439.6 MB	71.349	—
Inter-attribute redundancy only	334.9 MB	54.349	-23.8%
+ this method	295.2 MB	47.908	-32.9%

5.4 The yardstick for selection

Table 4 shows that the three stages of §3.5 are all needed.

Merely extending the candidate set lowered the code length at 2M points but raised it over the whole tile. The proxy was producing wrong decisions.

Adding the greedy verification (iii) uncovered two errors. Externalizing the return information (return number and the like) cost 2.273 bits per point, and the waveform deviation cost 0.036. Under the proxy the return information had looked like a 47% gain (1.298 for differencing against 0.688 for prediction); in reality the inner codec was coding it far more cheaply as part of the point record.

But (iii) alone is not enough. On a different file (TerraScan, 22.84M points) extending the candidates degraded the result from -4.7% to -2.4%: an attribute that used to go outside stayed in the container, and another switched to the wrong method. Since (iii) only puts back what went outside, it cannot catch this.

Adding (ii), measuring the marginal cost itself, makes that example -5.2%, better than the best before the extension (-4.7%). Over the whole tile it goes from 48.776 to 47.908 bits per point.

Even with (ii), (iii) is still needed. Because marginal costs are not additive, a wrong decision remained on the waveform deviation for the whole tile and on classification for another file, and (iii) caught them.

Replacing estimation by measurement alone recovered 3.1 bits per point. No new coding technique was added.

5.5 End to end

Table 5 gives the end-to-end result. The contribution of the attribute side is 6.441 bits per point, that is 9.0% of the original. It takes about 20 minutes including the marginal cost measurements, with 22.3 GB of peak memory.

Table 6: Effect of reordering the points on the code length. Airborne LiDAR, 2M points

	Acquis.	Geom.	Change
Geometry (LASzip)	22.219	28.232	+27.1%
Timestamp	2.263	14.546	+543%
Scan angle	0.191	0.742	+288%
Flight line number	0.001	0.271	large
Colour, 3 comp. (ours)	7.434	7.434	0%

5.6 Order is part of the representation

Table 6 shows that the arrangement of the points is by itself a factor in the code length. The LASzip geometry predictor is a sequential predictor along the scan line, and acquisition order is the input it handles best. Reordering the same point set into a geometry-derived order raises the code length by 27.1%. Timestamp, scan angle and flight line number are nearly monotone or constant in acquisition order, and reordering destroys that structure.

The colour code length of our method, by contrast, does not depend on the order, because (3) derives its order from the geometry rather than from the arrangement of the container.

There are two implications. First, when comparing coders the input must not be uniformly reordered. Reordering destroys the premise of one of the coders; it is not a levelling operation. Second, acquisition order and geometry-derived order are not mutually exclusive. Keeping the container in acquisition order while treating only the residual stream in a geometry-derived order uses the structure of both. A method that forces every point into a single order cannot do this.

6 The limits on the geometry side

The reason this paper addresses attributes is that little lossless headroom remains on the geometry side. We confirmed this from three independent angles.

6.1 Below the point spacing there is no context

Let $c \in \{1, \dots, 255\}$ be the occupancy byte of a node at depth ℓ and $\hat{c}$ the context available from its parent. When the leaf edge λ_ℓ is small enough relative to the median nearest neighbour distance $\bar{d}$ that each occupied node holds at most one point, c points to a single child, so $c = 2^k$ and $H(c) = H(k) \leq 3$ bits. In measurement, in the region where λ_ℓ falls below one quarter to one eighth of $\bar{d}$, all 13 datasets show $H(c) = 2.976$ –3.000 and sit on the equality side. Almost every neighbouring cell is empty, so there is nothing left to condition on.

6.2 Recovering the acquisition geometry lacks resolution

For a rotating LiDAR, returning Cartesian coordinates to spherical ones gives a large reduction. If the same

could be done for airborne LiDAR the geometry would become cheap, but that requires estimating the flight trajectory.

In our data the scan angle is retained only in 35 steps of one degree. At the maximum range of 400 m, one degree corresponds to about 7 m of cross-track position, which is far too coarse for trajectory estimation. Consecutive points in acquisition order are also a median of 0.284 m apart, which makes them a different thing from the neighbours at the 0.08 m nearest neighbour distance.

6.3 Sending height as a function of planimetric position does not help

The ground is close to a height field, so predicting Z from an XY neighbourhood ought to be cheap. If the decoder holds XY first, Proposition 1 says no side information is needed either.

Method	Z code length
LASzip	9.208
From the XY neighbourhood ($k = 1$)	9.022

The gain stops at 2%. There is a trap to note here. Searching for neighbours with a three-dimensional distance that includes Z brings the code length down to 7.444, but the decoder does not hold Z , so those neighbours are unusable. Restricted to two-dimensional neighbours the gain nearly vanishes.

7 Discussion

7.1 Where the bits are

This work first pursued improvements on the geometry side in four directions, and only after all of them closed did we notice that we had not looked at the larger half. The breakdown in which attributes hold 59% is not a special case. Figure 1(a) measures the same share over 15 files spanning airborne, automotive and terrestrial scanning: the median is 59.2%. The four files that carry no colour are exactly the four lowest shares, and every file that carries colour spends at least half of its code length on attributes. Attributes are the larger half for products that carry colour, which is most of what is distributed.

The same split separates where the gain comes from. Figure 1(b) measures the reduction this method obtains on each side of it: the geometry side is shorter on 13 of 15 files with a median of -9.2% , the attribute side on all 15 with a median of -18.0% . Taking the two sides together the reduction is -12.6% . Read alone, the geometry side would put this method level with an octree and slightly ahead of LASzip, and no ordering of the three would follow from it; the separation comes from the attribute side, and it comes from the order the geometry supplies.

7.2 Attributes divide into three by nature

Condition (8) gives a practical criterion that separates attributes into three groups. Attributes derived from orthoimagery are samples of a smooth field and can be predicted from a single nearest neighbour. Laser-derived attributes are a field with noise on top, and averaging several helps. Attributes that record the state of the device have no spatial structure, and there is nothing to do but send them as they are.

The distinction bears directly on the design of the coder, and it is also a quantitative tool for separating attributes that describe a property of a place from attributes that are a record of an individual shot.

7.3 Reach

The evaluation in this paper is limited to airborne LiDAR. We confirmed with a terrestrial laser scanner that it does not transfer directly to other acquisition modalities.

Returning to the acquisition representation does not transfer. For mobile LiDAR, returning Cartesian coordinates to spherical ones lowers the code length by 69%. A terrestrial scanner is tripod-mounted, so the sensor origin does not move, which lets us try the operation under the most favourable conditions. We used a public scan of a lecture hall (RIEGL VZ-400, 22.30M points). That the coordinate origin is the scanner position is confirmed by the regularity of the elevation steps (step 0.7017 mrad, dispersion 1.7%; displacing the origin by 50 cm raises the dispersion to 20–44%).

Error bound	Cartesian lattice	Spherical	Ratio
0.1 mm	13.934	18.741	+34%
1.5 mm	4.860	7.141	+47%
5.0 mm	2.295	3.613	+57%

What was -69% for mobile LiDAR becomes $+34$ – 57% here. Matching the error target to the angular resolution of the sensor does not change this.

The reason lies in the geometry of the scan. A rotating LiDAR leads with azimuth and follows with range: for each (elevation, azimuth) pair only the range varies, so spherical coordinates form a sparse lattice. In this terrestrial scan the range is nearly constant (1.43–3.16 m) and the points cling to the walls, so in spherical coordinates all three components vary. Returning to the acquisition representation pays only when that quantity forms a sparse lattice. That the transform is known and invertible is a necessary condition only.

Attribute coding is unverified elsewhere. We tried the attribute coder on the same terrestrial data and the reduction was 0.0%. That is not a failure of the method; the data carry nothing but coordinates and a static/dynamic flag. We could not obtain public terrestrial data carrying colour or intensity, so the attribute coder remains unverified outside airborne LiDAR.

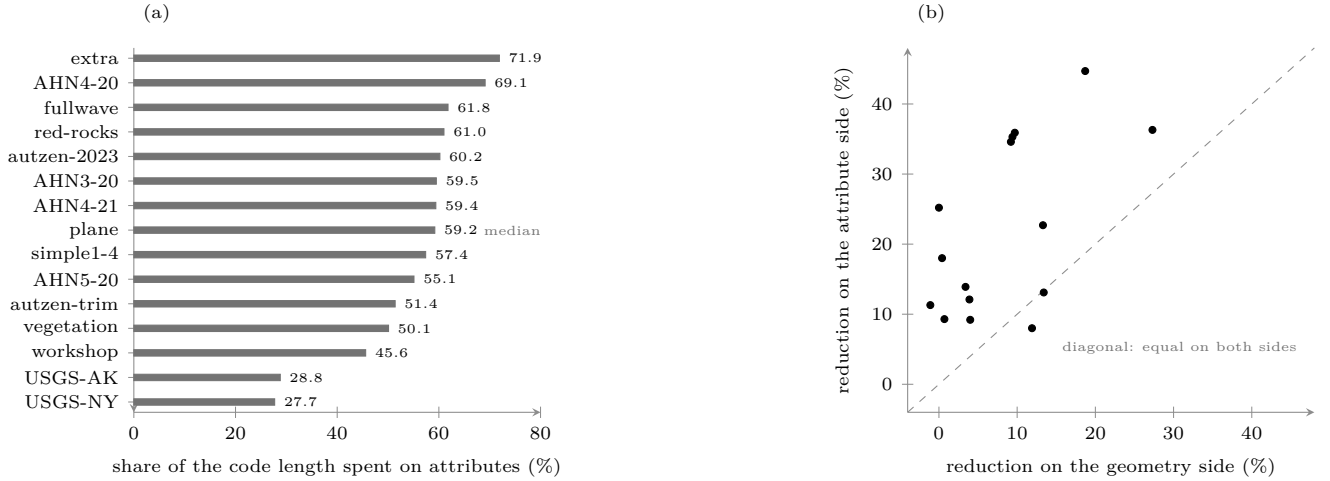


Figure 1: Where the bits are, measured over 15 airborne, automotive and terrestrial files — the centre 2×10^5 points of each, every row lossless, the baseline being LASzip over the same columns. (a) The share of the code length LASzip spends on attributes rather than on coordinates. The median is 59.2% and the range 27.7–71.9%. The four files that carry no colour are exactly the four lowest shares; every file that carries colour spends at least half of its code length on attributes. (b) The reduction this method obtains on each side of that split. A point above the diagonal is a file on which the attribute side gives more than the geometry side, which is 13 of the 15. The geometry side is shorter on 13 files with a median of -9.2% ; the attribute side is shorter on all 15 with a median of -18.0% .

7.4 Other limitations

First, the third stage of §3.5 is a one-at-a-time greedy check and does not exhaust combinations of attributes. Searching all methods to convergence on a small file showed the greedy procedure leaving up to 12.3% on the table (with one attribute, 0.6%; with a single candidate, 0%). The cost is proportional to the number of candidates, so on large data we have no choice but to stay greedy.

Second, computation. The search for relations between attribute pairs was originally $O(M^2N)$ and took 232 of the seconds spent on the whole tile. Choosing the partner on a sample of 0.25M points and remeasuring only the single chosen pair over all points reduced that to 15 seconds. The decisions did not change and the code length in fact improved. What remains is the marginal cost measurement, one write per attribute.

8 Conclusion

From the property that a point cloud container decodes geometry first, we showed that an order and a predictor determined by the coordinates alone are available without side information, and gave a lossless attribute coder built on that. Under identical conditions it is 21% smaller than LASzip and 42% smaller than the only lossless G-PCC configuration.

For the number of predictors we derived a condition expressed by the ratio of noise to smoothness, and confirmed that measurement divides the attributes into the three groups it predicts.

Candidate selection splits into three stages. External candidates share a coder and can be compared directly, but comparison against the container can only be done by measuring the marginal cost, and because marginal costs are not additive a verification afterwards is needed as well. Selecting by a proxy costs 2.3 bits per point on

one field, and making all three stages measured recovers 3.1 bits per point in total. That improvement was obtained without adding any new coding technique.

The arrangement of points is part of the representation just as coordinates and attributes are, and a careless reordering silently degrades the code length.

The evaluation is limited to airborne LiDAR. For terrestrial laser scanning we confirmed that returning to the acquisition representation is counterproductive. We could not obtain data from other modalities carrying attributes, so the reach of the attribute coder is unconfirmed. Acquisition order and geometry-derived order are not mutually exclusive: keeping the container in acquisition order while treating only the residual stream in a geometry-derived order uses both.

References

- [1] D. Meagher, “Geometric modeling using octree encoding,” *Computer Graphics and Image Processing*, vol. 19, no. 2, pp. 129–147, 1982.
- [2] R. Schnabel and R. Klein, “Octree-based point-cloud compression,” *Symposium on Point-Based Graphics*, pp. 111–120, 2006.
- [3] MPEG 3DG, *G-PCC Codec Description*, ISO/IEC JTC1/SC29/WG7, 2023. Reference implementation mpeg-pcc-tmc13 v23.0.
- [4] R. L. de Queiroz and P. A. Chou, “Compression of 3D point clouds using a region-adaptive hierarchical transform,” *IEEE Trans. Image Processing*, vol. 25, no. 8, pp. 3947–3956, 2016.
- [5] M. Isenburg, “LASzip: Lossless compression of LiDAR data,” *Photogrammetric Engineering & Remote Sensing*, vol. 79, no. 2, pp. 209–217, 2013.
- [6] H. S. Malvar, G. J. Sullivan, and S. Srinivasan, “Lifting-based reversible color transformations for image compression,” *Proc. SPIE*, vol. 7073, 2008.
- [7] G. M. Morton, *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*, IBM Ltd., Ottawa, 1966.
- [8] Actueel Hoogtebestand Nederland (AHN4), geotiles.citg.tudelft.nl.